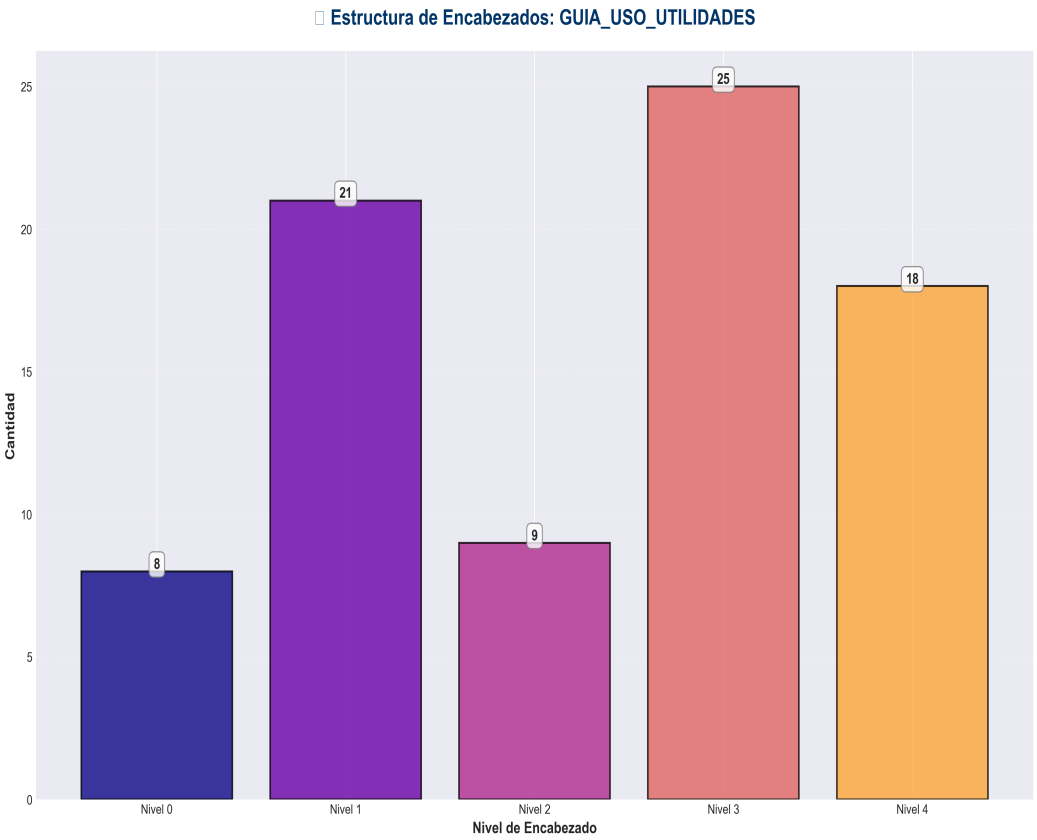
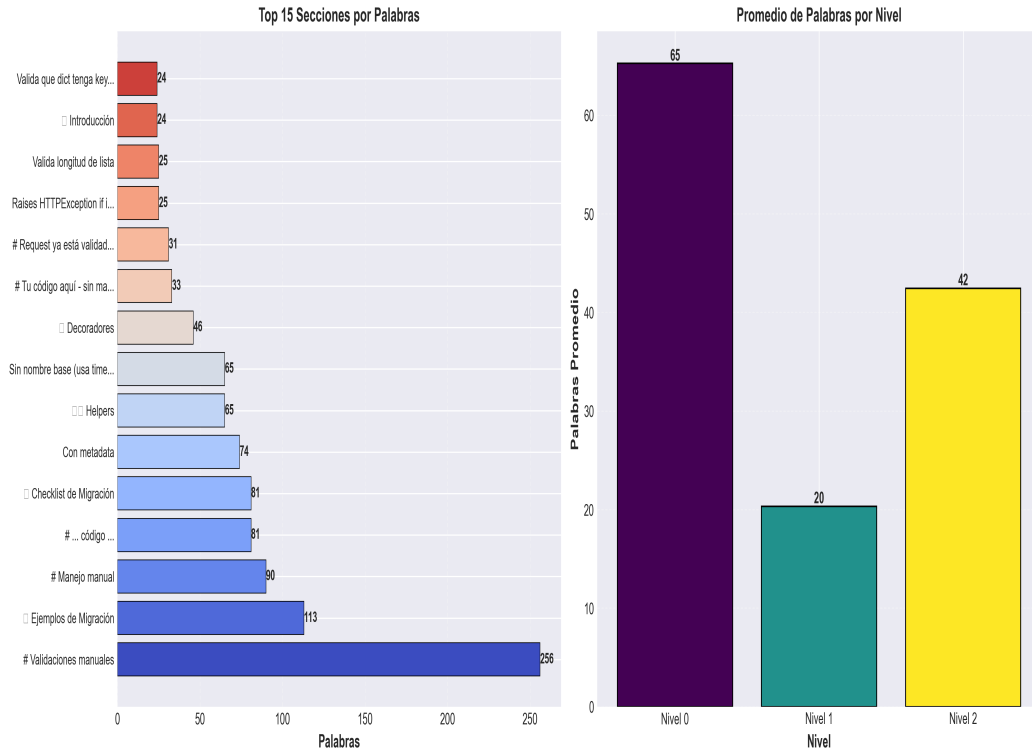


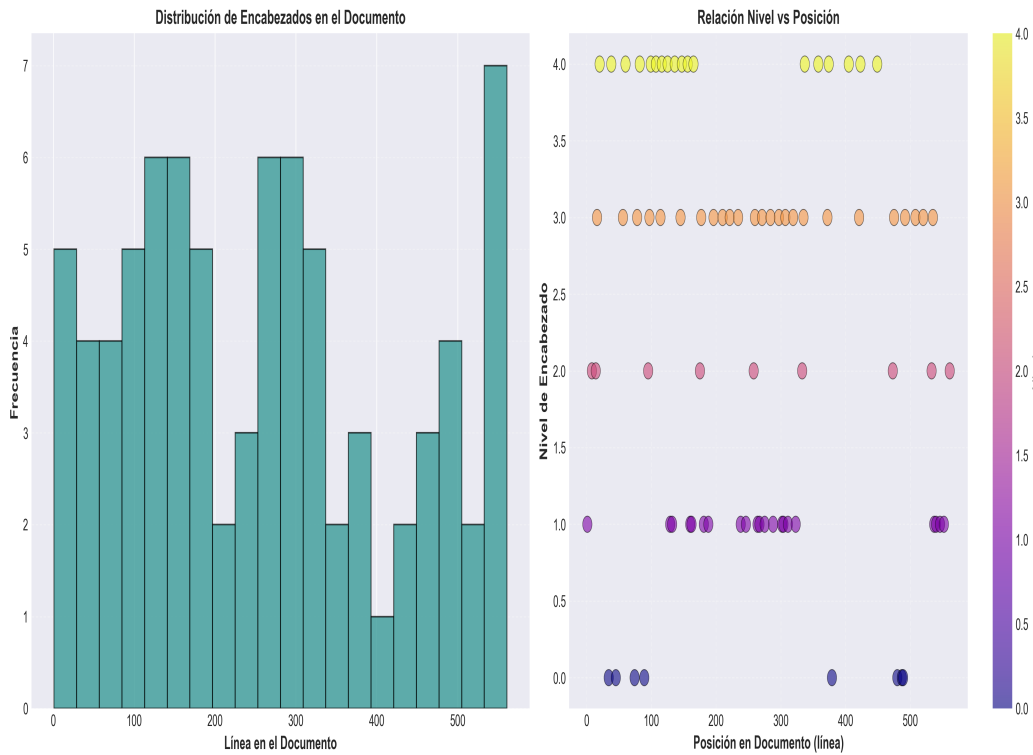
Análisis Adicionales



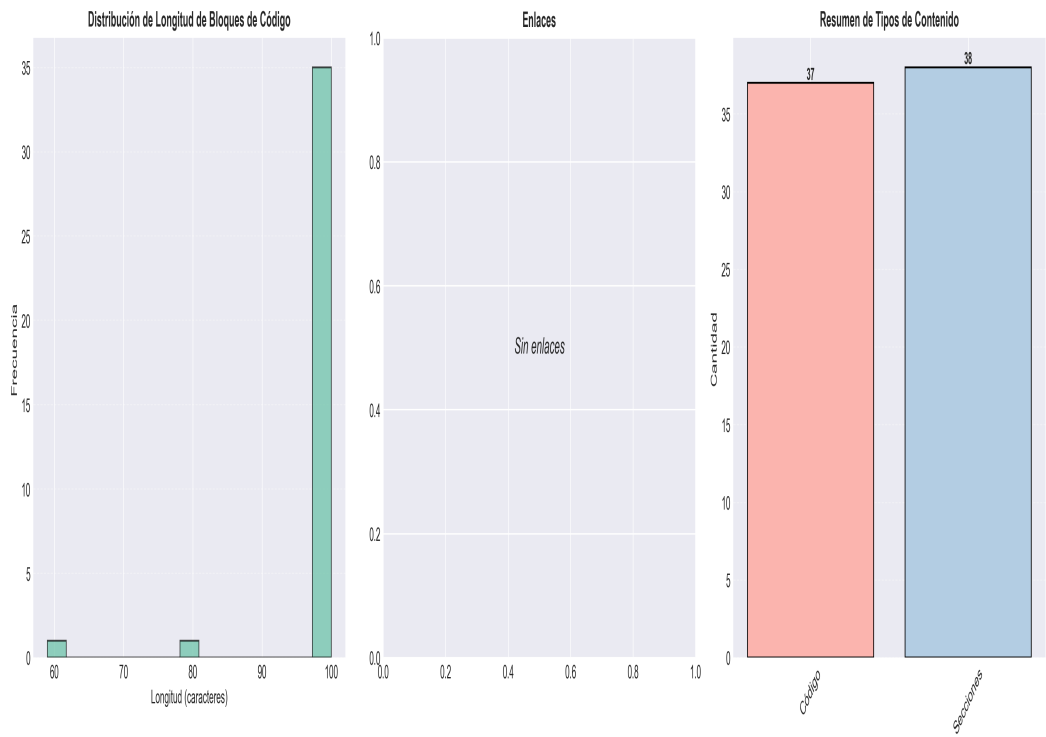
□ Análisis de Complejidad: GUIA_USO_UTILIDADES



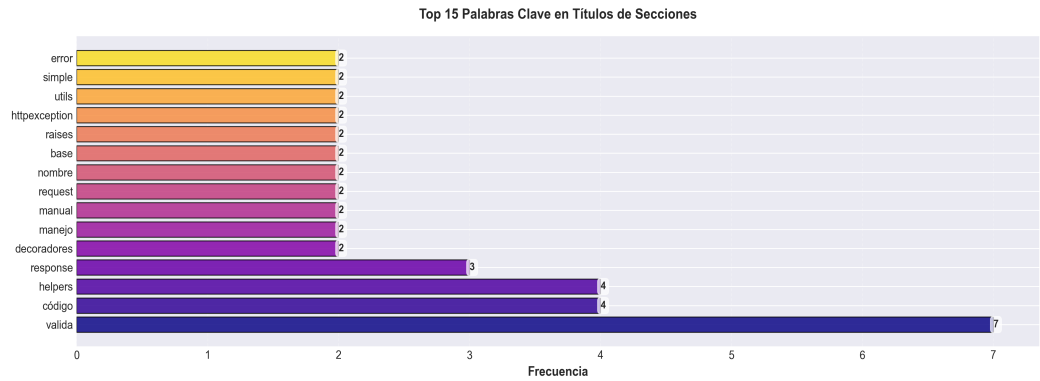
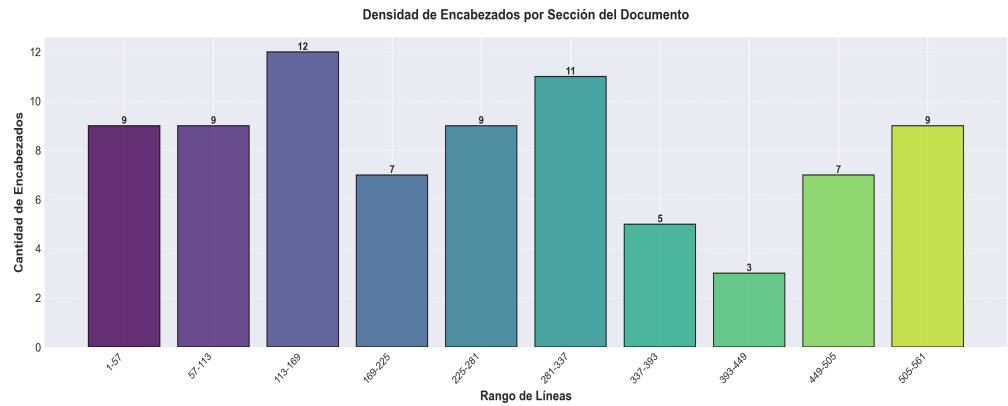
□ Análisis Temporal y Distribución: GUIA_USO_UTILIDADES



▣ Análisis Detallado de Contenido: GUIA_USO_UTILIDADES



□ **Análisis de Densidad y Frecuencia: GUIA_USO_UTILIDADES**



Resumen Ejecutivo: Este documento contiene 1,331 palabras distribuidas en 579 líneas, con 81 encabezados organizados en 38 secciones principales.

Estadísticas del Documento

Métrica	Valor
Total de líneas	579
Total de palabras	1,331
Total de caracteres	13,955
Encabezados	81
Bloques de código	37
Tablas	0
Enlaces	0
Imágenes	0
Secciones principales	38

Índice de Secciones

Nivel	Título	Línea
1	■ Guía de Uso de Utilidades	1
2	■ Introducción	8
2	■ Decoradores	14
0	# Tu código aquí - sin manejo de errores manual	34
0	# ... código ...	45
0	# Request ya está validado	74
0	# Request y response serán logueados automáticamente	89
2	■■ Helpers	95
1	Con nombre base	129
1	Sin nombre base (usa timestamp)	132
1	Valida y normaliza	160
1	Raises HTTPException if invalid	162
2	■ Response Utils	175
1	Respuesta simple	181
1	Con metadata	188
1	Error simple	238
1	Error con detalles	246
2	■ Validation Helpers	258
1	Valida que no esté vacío	264
1	Permite whitespace	267
1	Valida longitud	275
1	Valida longitud de lista	288
1	Valida que archivo exista	302
1	Raises HTTPException 404 if not found	304
1	Valida que dict tenga keys requeridas	311
1	Valida que valor esté en lista permitida	323
2	■ Ejemplos de Migración	333
0	# Validaciones manuales	379
2	■ Mejores Prácticas	473

0	# Código sin try/except	480
0	# ... código ...	487
0	# Manejo manual	489
2	■ Referencia Rápida	533
1	Decoradores	537
1	Helpers	540
1	Response Utils	546
1	Validation Helpers	552
2	■ Checklist de Migración	561

Resumen del Contenido

■ Guía de Uso de Utilidades

****Versión****: 2.0.0

****Fecha****: 2025-01-27

■ Introducción

Esta guía proporciona ejemplos prácticos de cómo usar las utilidades reutilizables creadas para mejorar la calidad y consistencia del código.

■ Decoradores

`@handle_route_errors``

Manejo consistente de errores en rutas API.

```
from api.decorators import handle_route_errors
```

```
from fastapi import APIRouter, Request
```

```
router = APIRouter()
```

```
@router.post("/store")
```

```
@handle_route_errors("store_episode", lambda s: s.is_available(),  
"memory")
```

```
async def store_episode(
```

```
request: MemoryStoreRequest,
```

```
req: Request,
```

```
service: MemoryService = Depends(get_memory_service)
```

```
):
```

Tu código aquí - sin manejo de errores manual

```
return service.store(request.episode)
```

```
@router.post("/store")
```

```
async def store_episode(request, req, service):  
    try:  
        if not service.is_available():  
            raise HTTPException(status_code=503, detail="Service not  
available")
```

... código ...

```
except HTTPException:  
    raise  
except ValueError as e:  
    logger.warning(f"Validation error: {e}")  
    raise HTTPException(status_code=400, detail=str(e))  
except Exception as e:  
    logger.error(f"Error: {e}", exc_info=True)  
    raise HTTPException(status_code=500, detail=str(e))
```

`@validate_request`

Validación de requests antes de ejecutar la ruta.

```
from api.decorators import validate_request  
  
def validate_message(request: ChatRequest) -> tuple[bool,  
Optional[str]]:  
    if not request.message.strip():  
        return False, "Message cannot be empty"  
    if len(request.message) > 10000:  
        return False, "Message too long (max 10000 characters)"  
    return True, None  
  
@router.post("/chat")  
@validate_request(validate_message)  
async def chat(request: ChatRequest):
```

Request ya está validado

```
    return service.chat(request.message)
```

`@log_request`

Logging de requests y responses.

```
from api.decorators import log_request
```

```
@router.post("/process")
```

```
@log_request(log_request_body=True, log_response=False)
```

```
async def process(request: Request, data: ProcessRequest):
```

Request y response serán logueados automáticamente

```
    return process_data(data)
```

■■ Helpers

Request Helpers

[Nota: El documento original tiene 579 líneas. Se muestra un resumen de las primeras 100 líneas.]